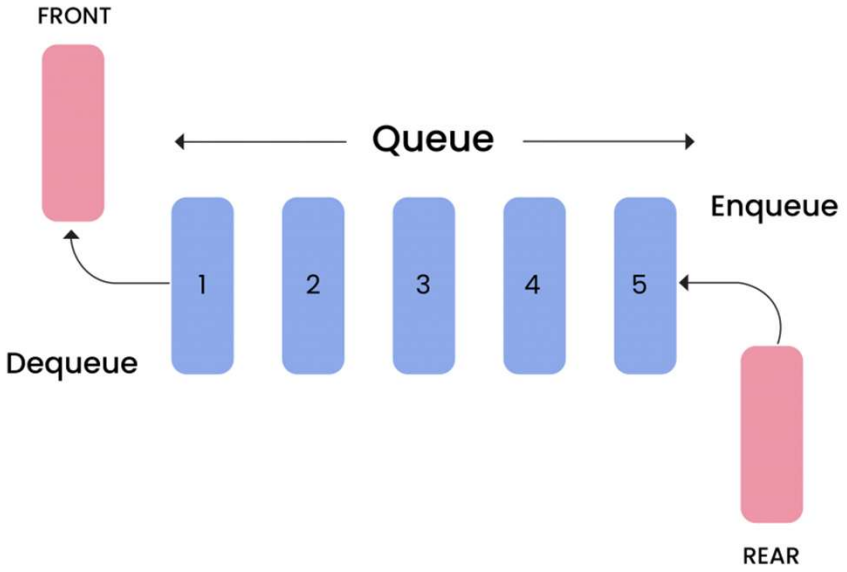
Struktur Data

Queue

masai



Definisi

Queue
Representasi Queue

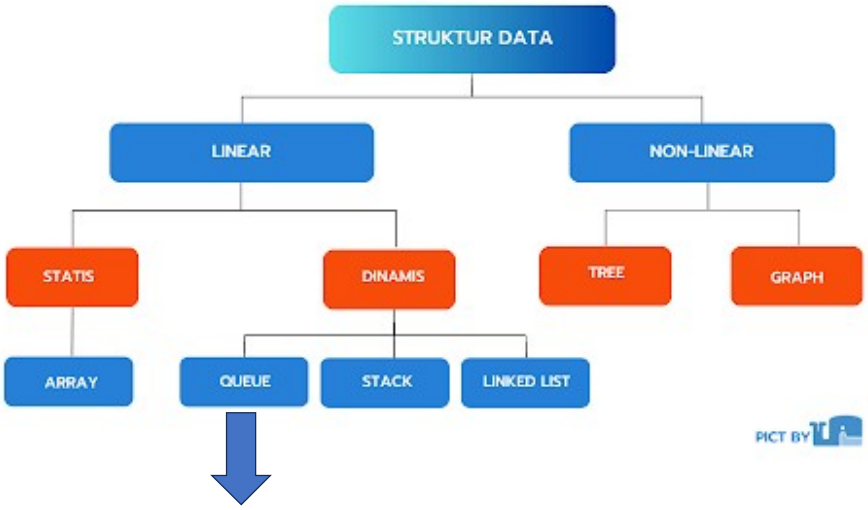
Objek Pembelajaran

- Memahami definisi, jenis, representasi, operasi, dan penerapan struktur data Queue
- Mampu memilih struktur data Queue yang tepat untuk merepresentasikan data.
- Mampu menyelesaikan masalah dengan menggunakan struktur data Queue secara efektif dan efisien.
- Meningkatkan kemampuan bernalar kritis, kreatif, mandiri, dan bergotong royong.

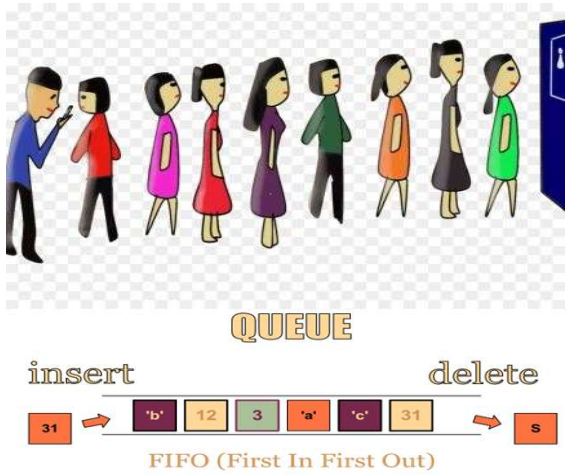
Keywords

Algoritma, Struktur data, Queue, AddQueue, DelQueue

DEFINISI



List linier yang dikenali elemen pertama (HEAD) dan elemen terakhirnya (TAIL), menggunakan prinsip kerja FIFO (First In First Out). (Liem, 2008).

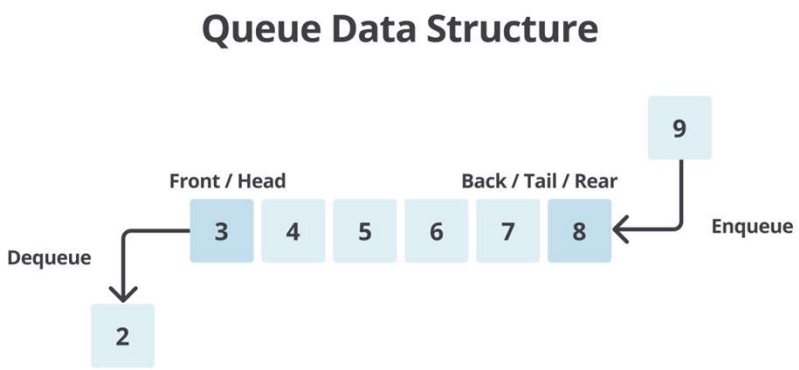


Queue memiliki peran yang penting dalam berbagai aplikasi dan algoritma. Salah satu fungsi utamanya adalah mengatur dan mengelola antrean tugas atau operasi secara efisien. Dalam sistem komputasi, ia digunakan untuk menangani tugas-tugas seperti penjadwalan proses, antrean pesan, dan manajemen sumber daya.

Konsep Queue

Queue adalah list linier yang :

1. Dikenali elemen pertama **(HEAD)** dan elemen terakhirnya **(TAIL)**
2. Aturan penyisipan dan penghapusan elemennya didefinisikan sebagai berikut:
    1. Penyisipan selalu dilakukan setelah elemen terakhir
    2. Penghapusan selalu dilakukan pada elemen pertama
    3. Satu elemen dengan yang lain dapat diakses melalui informasi NEXT



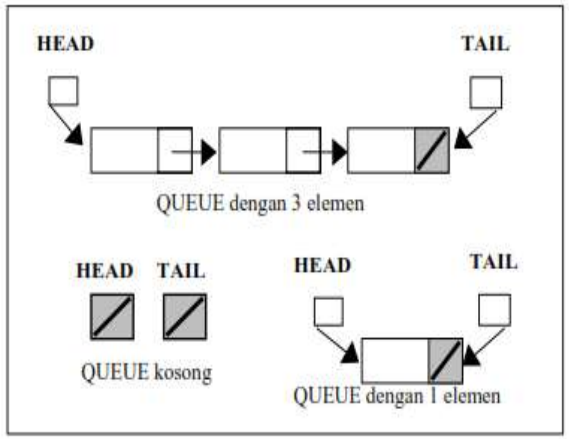
Struktur data ini banyak dipakai dalam informatika, misalnya untuk merepresentasikan:

- Antrian job yang harus ditangani oleh sistem operasi
- Antrian dalam dunia nyata

Konsep Queue

Secara lojik, sebuah QUEUE dapat digambarkan sebagai list linier yang setiap elemennya adalah:

```
Type ElmtQ :<Info : InfoType, Next : address>
```



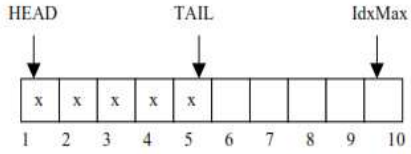
InfoType ⇒ sebuah type terdefinisi yang menentukan informasi yang disimpan pada setiap elemen Queue
Address ⇒ 'alamat' dari elemen
Alamat elemen pertama (HEAD)
Alamat elemen terakhir (TAIL)
Maka:
Jika Q adalah Queue dan P address, penulisan untuk Queue adalah:

Head(Q), Tail(Q), Info(Head(Q)), Info(Tail(Q))

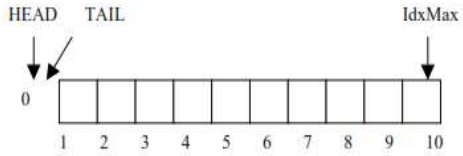
Implementasi Queue

Memori tempat penyimpan elemen adalah sebuah tabel dengan indeks **1..IdxMax**. IdxMax dapat juga "dipetakan" ke kapasitas Queue. Representasi field Next : jika i adalah **"address"** sebuah elemen, maka suksesor i adalah Next dari elemen Queue

Alternatif I :
Tabel dengan hanya representasi TAIL adalah indeks elemen terakhir, HEAD selalu diset sama dengan 1 jika Queue tidak kosong. Jika Queue kosong, maka HEAD=0.
Ilustrasi Queue tidak kosong, dengan 5 elemen :



Ilustrasi Queue kosong:



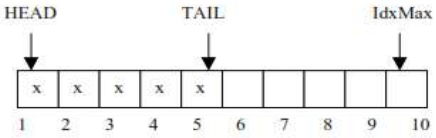
Jika masih ada tempat kosong untuk algoritma **penambahan elemen** dengan "memajukan" TAIL

Untuk penghapusan elemen jika Queue tidak kosong: ambil nilai elemen HEAD, geser semua elemen mulai dari HEAD+1 sampai dengan TAIL (jika ada), kemudian TAIL "mundur".

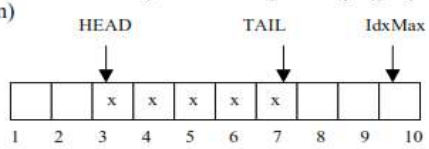
Implementasi Queue

Memori tempat penyimpan elemen adalah sebuah tabel dengan indeks **1..IdxMax**. IdxMax dapat juga "dipetakan" ke kapasitas Queue. Representasi field Next : jika i adalah **"address"** sebuah elemen, maka suksesor i adalah Next dari elemen Queue

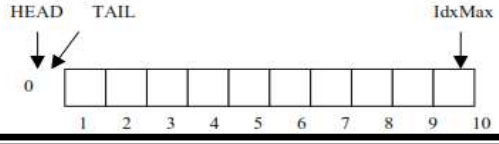
Alternatif II :
Tabel dengan representasi HEAD dan TAIL, HEAD "bergerak" ketika sebuah elemen dihapus jika Queue tidak kosong. Jika Queue kosong, maka HEAD=0.
Ilustrasi Queue tidak kosong, dengan 5 elemen, kemungkinan pertama HEAD "sedang berada di posisi awal:



Ilustrasi Queue tidak kosong, dengan 5 elemen, kemungkinan pertama HEAD tidak berada di posisi awal. Hal ini terjadi akibat algoritma penghapusan yang dilakukan (lihat keterangan)



Ilustrasi Queue kosong:



Jika masih ada tempat kosong untuk algoritma **penambahan elemen** dengan "memajukan" TAIL

Untuk penghapusan elemen jika Queue tidak kosong: ambil nilai elemen HEAD "maju", kasus khusus untuk Queue dengan keadaan awal berelemen 1, yaitu menyesuaikan HEAD dan TAIL dengan DEFINISI

Implementasi Algoritma dan Bahasa